

SyloPay BNPL Smart Contract Documentation

Stellar Soroban Smart Contract Architecture & API Specification

1. Overview & Architecture

The SyloPay BNPL contract enables decentralized Buy Now, Pay Later credit terms directly on-chain using the Stellar Soroban network and USDC. It empowers merchants to offer structured installment payment plans to customers, securing agreements and enforcing transparent states directly via Soroban ledger storage.

Modular Architecture

The contract is structured according to the industry's best-practice modular hierarchy to maximize readability, maintainability, and clean separation between test and production code:

- **lib.rs** — Main contract entrypoint and implementation of the external `#[contractimpl]` functions.
- **types.rs** — Storage models, structures, and status Enums decorated with `#[contracttype]`.
- **utils.rs** — Universal shared utilities and formatting helpers.
- **test.rs** — Isolated unit test suite compiled conditionally with `#[cfg(test)]`.

2. Storage Strategy & Gas Optimization

Soroban uses state storage pricing based on three distinct storage classes. The SyloPay contract implements a hybrid model designed to minimize transaction gas fees:

- **Instance Storage** — Holds global configurations like the Admin address and the global contract counter. This data is automatically loaded when invoking the contract, reducing reading overhead.
- **Persistent Storage** — Holds heavy records like individual `ContratoBNPL` structures and customer contract list registries. This storage class survives lease cycles, guaranteeing user data is never garbage collected.

3. Data Structures & Data Models

3.1 Status Enums

Enums represent the exact state machine of contracts and individual installments:

- **StatusParcela** — `Pendente`, `Paga`, `Vencida`.
- **StatusContrato** — `Ativo`, `Concluido`, `Inadimplente`.

3.2 Structs

Production data types representing records inside storage:

Parcela Struct Definition:

```
pub struct Parcela {  
    pub numero: u32,  
    pub valor_usdc: i128,  
    pub vencimento: u64,  
    pub status: StatusParcela,  
    pub tx_hash: String,  
    pub pago_em: u64,  
}
```

ContratoBNPL Struct Definition:

```
pub struct ContratoBNPL {  
    pub id: String,  
    pub merchant: Address,  
    pub cliente: Address,  
    pub valor_total: i128,  
    pub num_parcelas: u32,  
    pub parcelas: Vec,  
    pub status: StatusContrato,  
    pub criado_em: u64,  
}
```

4. Smart Contract Core API

4.1 State Modifying Functions

- **initialize(env: Env, admin: Address)** — Initializes the contract instance settings. Can only be executed once. Sets the Admin address and starts the global contract counter at 0.
- **criar_contrato(env: Env, merchant: Address, cliente: Address, valor_total: i128, num_parcelas: u32)** → **String** — Registers a new BNPL contract on-chain. Fails if the client signature authorization is missing. Generates a unique ID (e.g., BNPL-0001), calculates equal installment allocations and 30-day staggered deadlines, and publishes a BNPL_NEW event.
- **pagar_parcela(env: Env, contrato_id: String, numero_parcela: u32, tx_hash: String)** — Processes payment validation for a specific installment. Fails if the client signature authorization is missing. Updates status to Paga, saves the transaction hash and payment timestamp, and completes the contract if all installments are settled.
- **marcar_inadimplente(env: Env, contrato_id: String)** — Flags a contract as delinquent due to overdue payments. Fails if the contract Admin signature authorization is missing. Sets missed installments to Vencida, sets contract status to Inadimplente, and publishes a BNPL_DEF event.

4.2 Read-Only Query Functions

- **status_contrato(env: Env, contrato_id: String) -> ContratoBNPL** — Returns the complete data struct for a specific contract.
- **listar_contratos_cliente(env: Env, cliente: Address) -> Vec** — Returns a list of all contract IDs associated with a specific customer account.
- **obter_admin(env: Env) -> Address** — Returns the initialized contract administrator's address.
- **total_contratos(env: Env) -> u32** — Returns the total number of BNPL contracts registered on the platform.

5. Security & Authorization Matrix

The matrix below defines the access control policies enforced at the smart contract level:

Function	Required Auth	Enforcement Mechanism
initialize	None (One-time limit)	Checks instance storage for ChaveStorage::Admin presence
criar_contrato	cliente signature	Explicit cliente.require_auth() verification
pagar_parcela	cliente signature	Explicit contrato.cliente.require_auth() verification
marcar_inadimplente	admin signature	Explicit admin.require_auth() verification
status_contrato	Public	Read-only access
listar_contratos_cliente	Public	Read-only access